

Functional Analysis — Weather-Aware Voice Agent

Deliverable 1: the "think before you build" document — what the customer wants, what the system needs, how it decides, and where it can break.

1. Customer goal

"I'd love an AI voice agent I can call to check the weather in my town. If it's going to be cold today (below 10 °C / 50 °F), text me a reminder to bring a coat."

Restated plainly: a phone number the customer can call that (a) tells them the current weather for a town they name, and (b) if it's below 10 °C, sends them a text reminding them to bring a coat.

Success = caller hears a correct, natural weather answer, and *only when it's cold* receives a coat reminder by text (email if text can't be delivered).

2. Inputs

Input	Source	Notes
Town name	Spoken by caller, captured by the voice agent	Free text — must handle misspellings / unknown towns
Current temperature	Weather API (Open-Meteo)	Needs coordinates, so town is geocoded first
Cold threshold	Fixed business rule: < 10 °C	Configurable in code; 10.0 exactly is <i>not</i> cold
Reminder recipient	Caller's phone number (or a configured demo number)	Twilio trial can only send to <i>verified</i> numbers

3. Components

- Voice agent (Retell)** — answers the call, understands "what's the weather in X", speaks the reply. Calls out to our function for the actual data.
- Mid-call function** — `/api/weather` (serverless) — geocodes the town, fetches current weather, applies the cold rule, sends the reminder if needed, returns a spoken sentence to the agent.
- Weather API (Open-Meteo)** — free, no key: geocoding + current temperature.
- Notification service (Twilio SMS, Resend email fallback)** — delivers the coat reminder.
- Post-call function** — `/api/call-webhook` (serverless) — records the call after it ends (idempotent write, with a dead-letter queue for failures).
- Shared store (Upstash Redis)** — remembers processed call IDs, the call log, and the dead-letter queue across stateless serverless invocations.

4. Trigger logic (the decision flow)

```

Caller: "What's the weather in <town>?"
|
v
Agent calls /api/weather { town }
|
+- geocode(town) --> not found? --> agent: "I couldn't find that town, try another"
|
v
fetch current temperature
|
v
is temperature < 10 C ?
|
YES                                NO
|                                |
send "bring a coat" text        (skip - no reminder)
|                                |
v                                v
agent: "It's 4 C in Hobart and cloudy - I've texted you to bring a coat."
agent: "It's 19 C in Dewsbury and clear - no coat needed."

```

After the call ends, Retell posts a `call_ended` event to `/api/call-webhook`, which writes one record for the call (see failure handling below).

5. Edge cases & failure modes

This is the part the role cares about most — what happens when things go wrong.

Case	What happens	Handling
Unknown / misspelled town	Geocoder returns no result	Friendly re-prompt: "I couldn't find that town, try a nearby one" (HTTP 200, not a crash)
Temperature exactly 10.0 °C	Boundary of the rule	Defined explicitly: <code>< 10</code> → 10.0 is <i>not</i> cold. A choice, not an accident
Weather API down / slow	fetch fails or times out	Caught; agent says "I couldn't reach the weather service, try again" — the call still ends gracefully
SMS fails / number unverified	Twilio returns an error	Automatic fallback to email; if both fail the weather answer <i>still</i> succeeds
Duplicate post-call webhook	Retell retries <code>call_ended</code>	Idempotency on <code>call_id</code> — the second copy is ignored, so no duplicate records
Database write fails	Store unavailable / bad data	Dead-letter queue — the event is parked for retry, never silently dropped
<code>call_ended</code> never arrives	Webhook lost / call crashed	Missing-webhook detection — <code>call_started</code> is recorded; <code>/api/reconcile</code> flags calls that never ended

6. Deliberate scope cuts (prototype, not launch)

- One weather provider, no caching, no retries-with-backoff on the API call.
- Reminder recipient defaults to a configured demo number rather than a full "collect and verify the caller's number" flow.
- Reconcile is an on-demand endpoint, not a running cron.
- Idempotency store falls back to in-memory when Redis isn't configured.

Each of these is a conscious trade-off for a ~90-minute prototype, listed here so the boundary between "built" and "would build next" is explicit.